

Project Title

FoodSense: A Visual Classifier and Nutrition Chatbot

Abstract:

This project presents an AI-powered food classification and assistant application that enables users to upload images of food and receive instant categorization along with intelligent responses to food-related queries. By combining the CLIP model for visual recognition and a language model (Mistral-7B via Open Router) for generating natural language answers, the app serves both as a food identifier and a virtual food expert. The interface, built using Streamlit, allows users to interact with the system in a user-friendly environment. This project demonstrates the potential of combining vision and language models for real-world applications in health, nutrition, and food education.

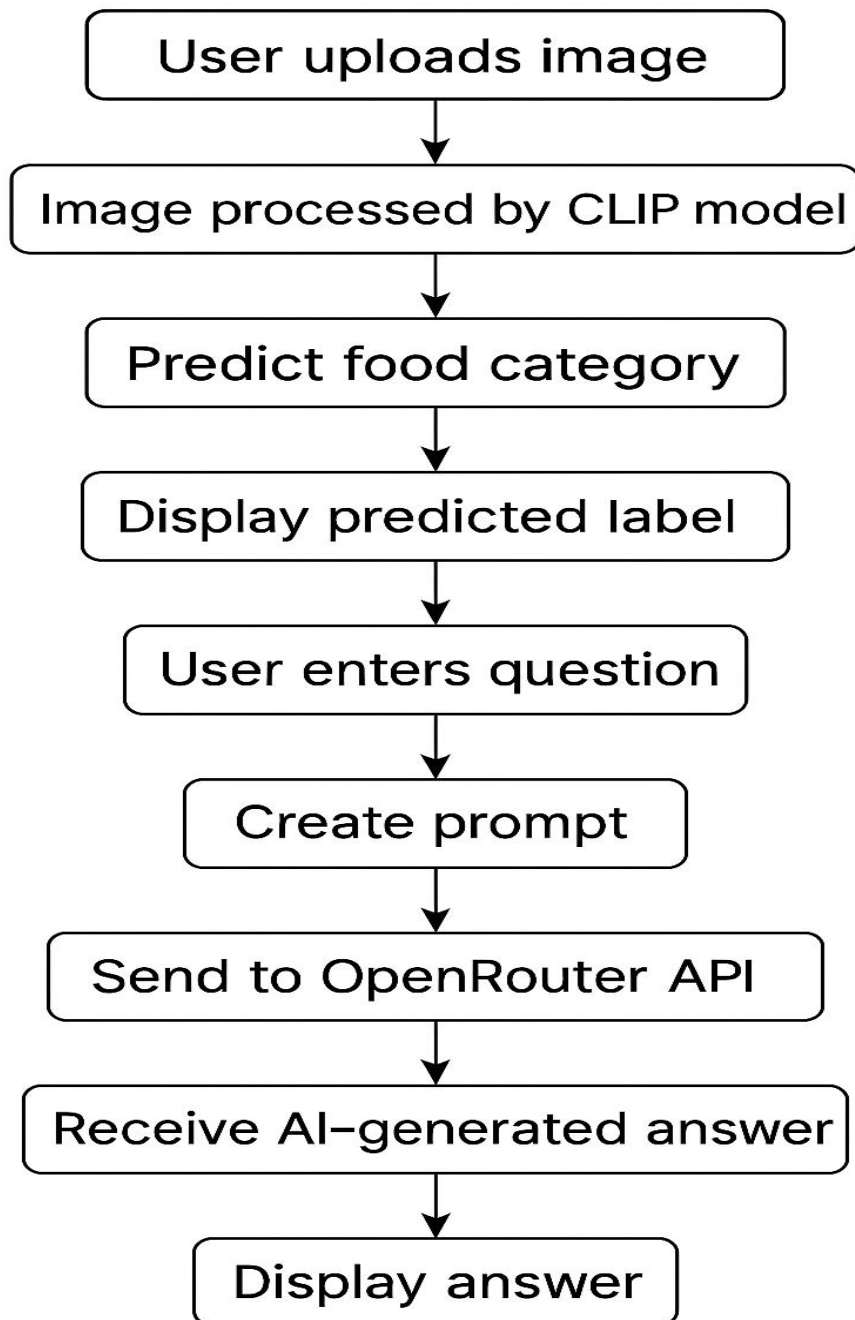
Objective:

The primary objective of this project is to develop an AI-based application that can automatically classify food items from images and provide intelligent, expert-like responses to user queries related to the identified food. The goal is to promote food awareness and healthy eating habits by using machine learning and natural language processing in a user-friendly web interface. The system aims to assist users in identifying whether the food is healthy or not, and to offer suggestions or nutritional tips accordingly.

Methodology:

The project begins with image classification using the CLIP model, which compares the uploaded food image with a list of predefined food categories such as fruits, vegetables, junk food, and desserts. Once the model identifies the most relevant category based on visual and textual similarity, the result is displayed to the user. After classification, the user can ask a question about the food, which is then sent along with the predicted label to a large language model (Mistral-7B) through the OpenRouter API. This model generates a friendly and informative response, simulating the expertise of a virtual food advisor. The entire process is handled through a Streamlit-based interface, which makes interaction smooth and intuitive. Finally, the application is deployed using PyNgrok, allowing it to be accessed through a public link for real-time use and demonstration.

Architecture Diagram:



Code:

```
from transformers import CLIPProcessor, CLIPModel
from PIL import Image
import torch
import numpy as np
from IPython.display import display
import openai
import glob
import os

labels = ["healthy food", "junk food", "fruit", "snack", "vegetable", "dessert"]

def classify_image(image: Image.Image):
    inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)
    outputs = model(**inputs)
    probs = outputs.logits_per_image.softmax(dim=1).detach().numpy()[0]
    best_idx = np.argmax(probs)
    return labels[best_idx], probs[best_idx]

import requests

def generate_response(classification, question):
    api_key="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"
    url = "https://openrouter.ai/api/v1/chat/completions"

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
        "HTTP-Referer": "https://your-project-or-email", # Optional but recommended (you can use your email)
    }
```

```

prompt = (
    f"This is a {classification}.\n\n"
    f"Question: {question}\n"
    f"Respond as a friendly food expert with reasoning and healthy tips."
)

```

```

data = {
    "model": "mistralai/mistral-7b-instruct", # Free and fast
    "messages": [
        {"role": "user", "content": prompt}
    ]
}

```

```

response = requests.post(url, headers=headers, json=data)

```

```

if response.status_code == 200:
    return response.json()["choices"][0]["message"]["content"]
else:
    print("Error:", response.status_code, response.json())
    return "Sorry, I couldn't process that."

```

```

!wget -nc http://data.vision.ee.ethz.ch/cvl/food-101.tar.gz
!tar -xzf food-101.tar.gz

```

```

!pip install streamlit transformers torch Pillow openai

```

```

code = ""
# Your full streamlit-based code goes here
# Example:
import streamlit as st
from transformers import CLIPProcessor, CLIPModel

```

```

from PIL import Image
import torch
import numpy as np
import requests

st.title("🍌 Food Image Classifier + Expert Advice")

labels = ["healthy food", "junk food", "fruit", "snack", "vegetable", "dessert"]
model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32")
processor = CLIPProcessor.from_pretrained("openai/clip-vit-base-patch32")

def classify_image(image):
    inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)
    outputs = model(**inputs)
    probs = outputs.logits_per_image.softmax(dim=1).detach().numpy()[0]
    best_idx = np.argmax(probs)
    return labels[best_idx], probs[best_idx]

def generate_response(classification, question):
    api_key="sk-or-
v14b249611338b7e707a87243b71403cf71678fa29280efbc43dbc6520e373dec7" # Your
OpenRouter key here

    url = "https://openrouter.ai/api/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json",
        "HTTP-Referer": "https://your-project-or-email",
    }

    prompt = f"""This is a {classification}.
    Question: {question}
    Respond as a friendly food expert with reasoning and healthy tips."""
    data = {

```

```

    "model": "mistralai/mistral-7b-instruct",
    "messages": [{"role": "user", "content": prompt}]
}

```

```

response = requests.post(url, headers=headers, json=data)
if response.status_code == 200:
    return response.json()["choices"][0]["message"]["content"]
else:
    return "Sorry, I couldn't get a response."

```

```

uploaded_file = st.file_uploader("Upload a food image", type=["jpg", "png", "jpeg"])
if uploaded_file:

```

```

    image = Image.open(uploaded_file).convert("RGB")
    st.image(image, caption="Uploaded Image", use_column_width=True)

```

```

label, confidence = classify_image(image)
st.markdown(f'### 🍷 Prediction: `{label}` ({confidence*100:.2f}% confidence)')

```

```

question = st.text_input("Ask a question about this food:")
if question:

```

```

    answer = generate_response(label, question)
    st.markdown("### 🗨️ GPT says:")
    st.markdown(answer)

```

```

'''

```

```

with open("app.py", "w") as f:

```

```

    f.write(code)

```

```

from pyngrok import conf

```

```

conf.get_default().auth_token =
"2vd2rjnPTGIGQvH49N0izwAHZmR_4mRgJhbHLEs6i9kRs5YdP"

```

```

!pip install pyngrok

```

```

from pyngrok import ngrok

```

```

# Kill any existing tunnels

```

```
ngrok.kill()

# Open new tunnel

public_url = ngrok.connect(8501)

print("🔗 Streamlit App URL:", public_url)# Start Streamlit !streamlit run app.py &> /dev/null &
```

Output:

